

E-Chronometraż: System Zarządzania Aeroklubem

Dokumentacja projektowa

Grupa projektowa:

Yahor Kalesnikau

Mateusz Brokos

Anton Homan

Szymon Tyliński

Kacper Wilgus

26 czerwca 2026

Spis treści

1	Wstęp i cel projektu	3
2	Skład grupy realizującej projekt	3
3	Przegląd istniejących rozwiązań	3
4	Analiza przypadków użycia	3
5	Analiza “User Story” (Opowieści Użytkownika)	4
5.1	Rola: Pilot (Użytkownik końcowy)	4
5.2	Rola: Administrator (Pracownik aeroklubu)	4
6	Analiza wymagań projektowych	4
6.1	Wymagania funkcjonalne i ich priorytety	4
6.2	Wymagania нефункционалне	5
7	Harmonogram i realizacja etapów	5
8	Analiza ryzyka na poziomie kamieni milowych	6
9	Specyfikacja techniczna zrealizowanego systemu	6
9.1	Architektura systemu i technologie	6
9.2	Diagram bazy danych (ERD)	7
9.3	Opis protokołów komunikacyjnych	7
9.4	Prototyp interfejsu użytkownika (MVP)	7
9.5	Repozytorium kodu	9
10	Przyjęte metody organizacji pracy	9

11 Zakres systemu i ograniczenia projektu	10
12 Role i uprawnienia użytkowników	10
13 Opis API aplikacji	10
14 Przepływy działania systemu	11
14.1 Logowanie użytkownika	11
14.2 Przegląd statków powietrznych przez pilota	11
14.3 Wybór terminu lotu	12
14.4 Aktualizacja cennika przez administratora	12
15 Walidacja danych i obsługa błędów	12
16 Testowanie systemu	12
17 Instrukcja uruchomienia projektu	13
18 Bezpieczeństwo systemu	14
19 Decyzje projektowe i uzasadnienie technologii	14
20 Podsumowanie realizacji wymagań	15
21 Możliwości dalszego rozwoju	16
22 Deklaracja wykorzystania narzędzi AI	16

1 Wstęp i cel projektu

Celem projektu jest stworzenie aplikacji webowej „E-Chronometraż”, służącej do cyfryzacji procesu zarządzania lotami w mniejszych aeroklubach. System ma zastąpić tradycyjne, papierowe dzienniki lotów (tzw. chronometraże), co pozwoli na wyeliminowanie błędów w ręcznych wyliczeniach kosztów oraz usprawnienie procesu rezerwacji statków powietrznych.

2 Skład grupy realizującej projekt

W realizację projektu zaangażowani byli następujący studenci, z przypisanymi rolami zgodnie z przyjętym harmonogramem:

- **Yahor Kalesnikau** – Scrum Master / PM (koordynacja prac i dokumentacja).
- **Mateusz Brokos** – Frontend Developer (Panel Pilota oraz Panel Administratora).
- **Anton Homan** – Frontend Developer (Panel Pilota oraz Panel Administratora).
- **Szymon Tyliński** – Backend Developer (API, autoryzacja JWT, logika kosztów).
- **Kacper Wilgus** – Baza danych i DevOps (model relacyjny i wdrożenie).

3 Przegląd istniejących rozwiązań

Obecnie mniejsze aerokluby borykają się z wyzwaniami wynikającymi z analogowego zarządzania:

- **Papierowe dzienniki:** Podatność na błędy ludzkie przy ręcznym sumowaniu minut lotu i mnożeniu przez stawki.
- **Utrudniona rezerwacja:** Konieczność kontaktu telefonicznego w celu sprawdzenia dostępności maszyny.
- **Brak centralnej bazy danych:** Trudności w szybkim generowaniu raportów finansowych.

4 Analiza przypadków użycia

Poniższa tabela przedstawia główne interakcje użytkowników z systemem, które stanowią podstawę logiki biznesowej aplikacji.

Aktor	Przypadek użycia	Opis
Pilot	Logowanie	Uzyskanie dostępu do danych osobistych i kalendarza.
Pilot	Rezerwacja samolotu	Wybór maszyny i terminu w interaktywnym widoku.
Administrator	Zarządzenie flotą	Dodawanie nowych jednostek i edycja istniejących.
Administrator	Zarządzenie cennikiem	Aktualizacja stawek za minutę lotu dla maszyn.
System	Autoryzacja ról	Weryfikacja uprawnień (Navigation Guards) do paneli.

Tabela 1: Główne przypadki użycia systemu E-Chronometraż.

5 Analiza “User Story” (Opowieści Użytkownika)

5.1 Rola: Pilot (Użytkownik końcowy)

- **US-P1 (Logowanie):** Jako pilot, chcę móc zalogować się do systemu, aby moje dane o lotach były bezpieczne.
- **US-P2 (Przegląd kalendarza):** Jako pilot, chcę widzieć interaktywny kalendarz lotów.
- **US-P3 (Rezerwacja):** Jako pilot, chcę zarezerwować wybrany samolot w wolnym terminie.

5.2 Rola: Administrator (Pracownik aeroklubu)

- **US-A1 (Zarządzanie flotą):** Jako administrator, chcę móc dodawać i edytować dane samolotów.
- **US-A2 (Edycja cennika):** Jako administrator, chcę móc aktualizować stawki za minutę lotu.
- **US-A3 (Rozliczenia lotów):** Jako administrator, chcę wprowadzać czas lotu, by system automatycznie wyliczył należność.

6 Analiza wymagań projektowych

6.1 Wymagania funkcjonalne i ich priorytety

Każde wymaganie funkcjonalne zostało poddane priorytetyzacji metodą MoSCoW, aby określić najważniejsze funkcjonalności z perspektywy wdrożenia (kamienie milowe).

ID	Opis wymagania	Priorytet
F1	System umożliwia logowanie i autoryzację dla ról Pilot oraz Administrator.	Krytyczny (Must)
F2	System prezentuje interaktywny kalendarz dostępności maszyn.	Krytyczny (Must)
F3	Użytkownik (Pilot) może dokonać rezerwacji wolnego terminu.	Krytyczny (Must)
F4	Administrator może zarządzać bazą samolotów i stawkami cennikowymi.	Wysoki (Should)
F5	System automatycznie wylicza koszt lotu (minuty $\times$ stawka).	Krytyczny (Must)

Tabela 2: Zestawienie wymagań funkcjonalnych oraz ich priorytetów.

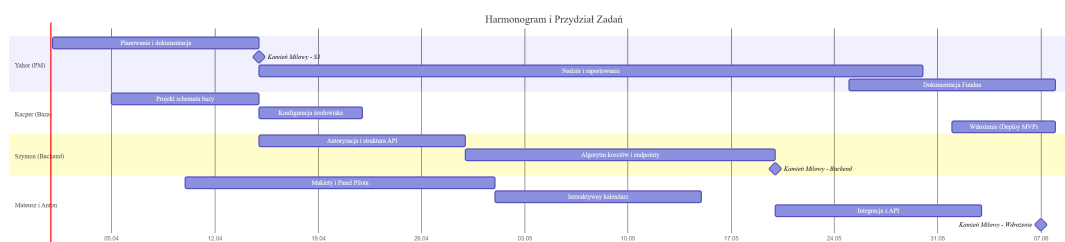
6.2 Wymagania niefunkcjonalne

ID	Opis wymagania
NF1	Bezpieczeństwo: Hasła są haszowane; komunikacja oparta na JWT.
NF2	Wydajność: Czas ładowania interfejsu poniżej 2 sekund.
NF3	Dostępność: System hostowany w chmurze, dostępny 24/7.

Tabela 3: Zestawienie wymagań niefunkcjonalnych.

7 Harmonogram i realizacja etapów

Aby udowodnić, że projekt wykonywany jest zgodnie z założeniami, zespół ściśle kontroluje wykorzystanie dostępnego "budżetu czasowego". Przebieg prac w czasie prezentuje poniższy schemat Gantta.



Rysunek 1: Harmonogram prac projektu i wykorzystanie budżetu czasowego (Schemat Gantta).

Realizacja projektu opiera się na dostarczaniu wyznaczonych **kamieni milowych** (ang. milestones). Proces weryfikacji zdobycia kamienia milowego odbywa się poprzez rygorystyczne podejście do testowania oprogramowania. Przykładowo, aby dany etap został uznany za zrealizowany, testy funkcjonalne muszą być przygotowane w 100% i przechodzić pomyślnie w co najmniej 40%.

8 Analiza ryzyka na poziomie kamieni milowych

Świadomi faktu, że aplikacja sama w sobie nie podlega ocenie bezpośrednio, główny nacisk w analizie ryzyka kładziemy na to, w jaki sposób reagujemy na problemy techniczne. Poniższa tabela przedstawia plany awaryjne (mitygacje) w sytuacji, gdy zespół nie zrealizuje określonego kamienia milowego w budżecie czasowym.

Kamień milowy	Zagrożenie / Scenariusz (Co jeśli etap się nie uda?)	Plan awaryjny (Mitygacja ryzyka)
M1: Moduł logowania	Niepowodzenie integracji systemu logowania lub błędy w autoryzacji (JWT).	Wyłączenie weryfikacji tokenów; zastosowanie tzw. autoryzacji "hardcoded" w Pinii (mockowanie ról do prezentacji UI).
M2: Baza danych API	Opóźnienia w produkcji kodu backendowego; niestabilność serwera lub relacji.	Niezależny rozwój UI przy użyciu plików JSON (Mock API) na froncie. Gotowość do wdrożenia na darmowym hostingu zastępczym.
M3: Rezerwacje lotów	Błędy w synchronizacji kalendarza dostępności w czasie rzeczywistym.	Zastąpienie interaktywnego kalendarza graficznego uproszczoną listą tekstową wolnych slotów czasowych.
M4 Wyliczanie kosztów lotów	Ryzyko niezrealizowania wymagania funkcjonalnego F5, czyli automatycznego wyliczania kosztu lotu na podstawie czasu lotu i stawki minutowej.	Zachowanie kluczowego elementu potrzebnego do późniejszego rozliczania, czyli modułu cennika minutowego powiązanego z typem statku powietrznego. W wariantcie awaryjnym administrator ma dostęp do aktualnych stawek, a koszt może zostać obliczony ręcznie według wzoru: $\text{czas lotu} \times \text{stawka za minutę}$. Dzięki temu system nadal wspiera proces rozliczeń, a jednocześnie pozostawia przygotowaną bazę do późniejszego dodania automatycznego zapisu i wyliczania należności.

Tabela 4: Kamienie milowe, scenariusze awaryjne i mitygacje ryzyk.

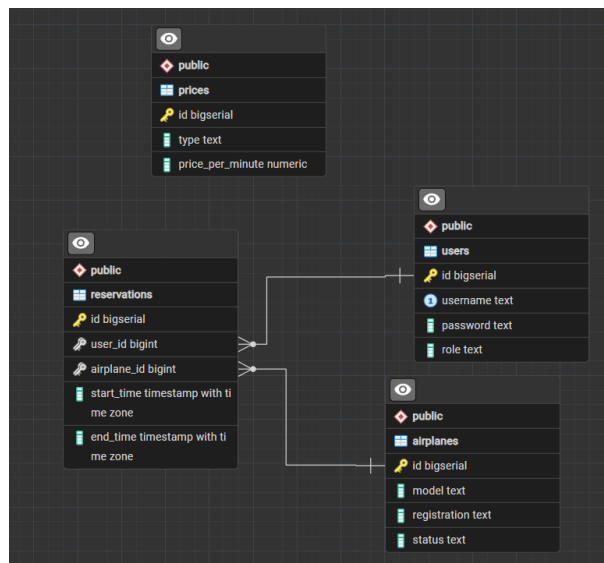
9 Specyfikacja techniczna zrealizowanego systemu

9.1 Architektura systemu i technologie

Aplikacja została zbudowana w architekturze klient-serwer:

- **Frontend:** Wykorzystano framework **Vue.js 3** z językiem **TypeScript**. Do zarządzania stanem wykorzystano bibliotekę **Pinia**, a za bezpieczeństwo tras odpowiada **Vue Router** z mechanizmem *Navigation Guards*.
- **Backend:** Serwer napisany w języku **Go** (standard REST). Autoryzacja realizowana przez tokeny **JWT**.
- **Baza danych:** Relacyjna baza **PostgreSQL**.

9.2 Diagram bazy danych (ERD)



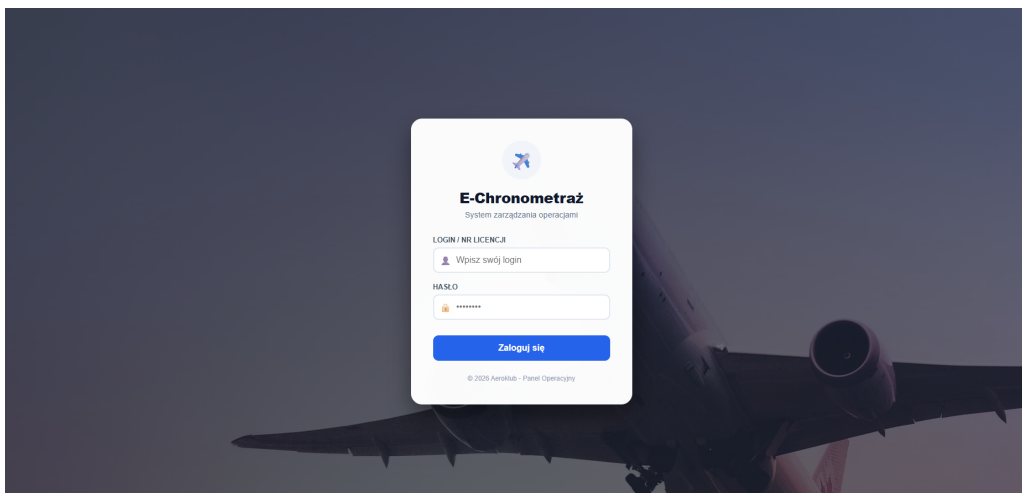
Rysunek 2: Diagram ERD bazy danych do testów

9.3 Opis protokołów komunikacyjnych

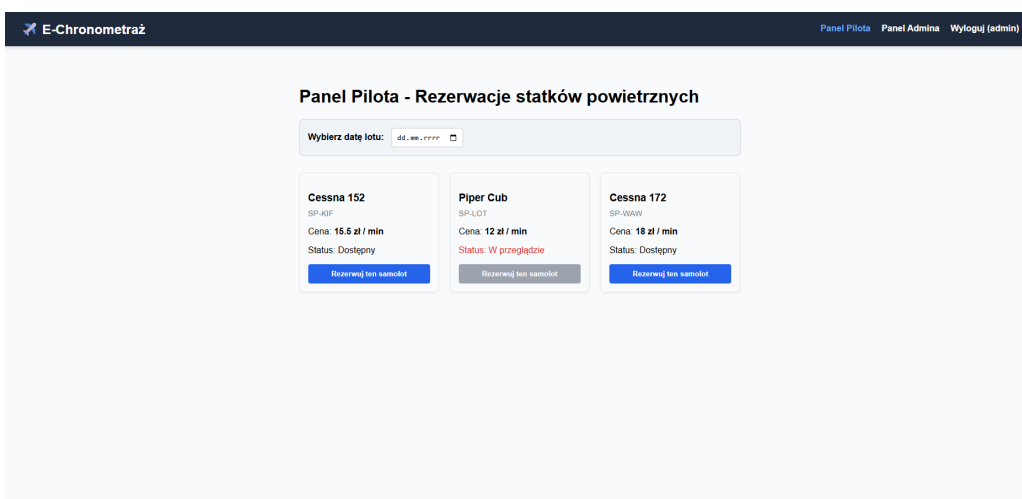
Komunikacja odbywa się przez zapytania HTTP (REST). Dane przesyłane są w formacie JSON. Autoryzacja wymaga nagłówków Bearer (JWT).

9.4 Prototyp interfejsu użytkownika (MVP)

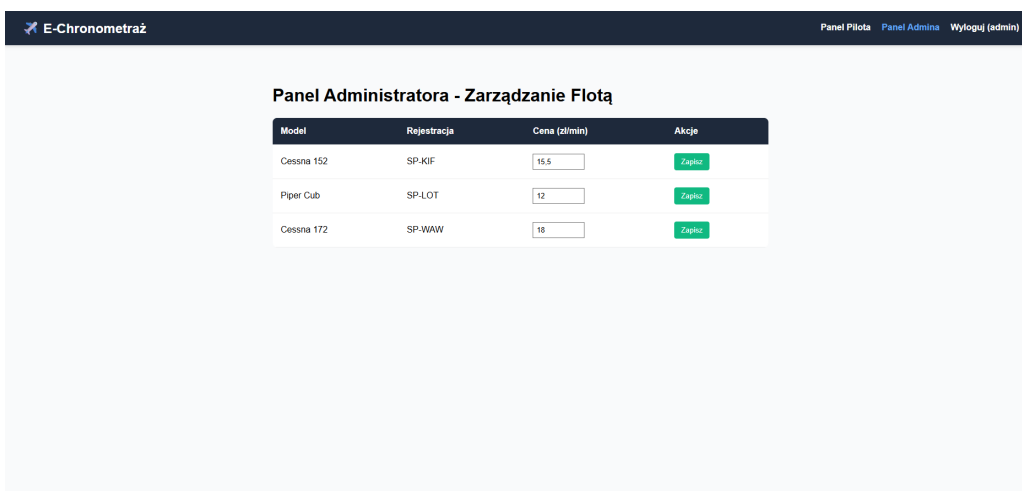
W ramach prac nad frontendem przygotowano już działające widoki aplikacji, zintegrowane z mockowanymi danymi (JSON) oraz systemem zarządzania stanem Pinia. Poniżej przedstawiono wybrane ekrany systemu:



Rysunek 3: Panel logowania do systemu E-Chronometraż.



Rysunek 4: Interaktywny kalendarz dostępności w Panelu Pilota.



Rysunek 5: Panel admina.

9.5 Repozytorium kodu

Kod źródłowy projektu dostępny jest pod adresem:

https://github.com/PWr-ISK-collaborative-project/Aeroklub_e-Chronometraz/tree/main

10 Przyjęte metody organizacji pracy

Praca opiera się na metodologii Agile. Komunikacja odbywa się przez Discord, a kod wersjonowany jest na GitHubie z Code Review (feature-branches). Zespół spotyka się na statusach typu Weekly.

11 Zakres systemu i ograniczenia projektu

System E-Chronometraż jest aplikacją webową przeznaczoną dla aeroklubów. Celem systemu jest uproszczenie obsługi podstawowych procesów związanych z zarządzaniem flotą, rezerwacją statków powietrznych oraz obsługą cennika lotów.

Aktualny zakres systemu obejmuje logowanie użytkowników, rozróżnienie ról pilota i administratora, wyświetlanie dostępnych statków powietrznych, prezentację stawek minutowych, panel pilota z wyborem daty lotu, panel administratora z możliwością edycji cennika oraz pomocniczy widżet prognozy pogody dla lotniska Szymanów.

Projekt nie obejmuje jeszcze pełnego systemu trwałego zapisu rezerwacji w bazie danych. W obecnym stanie rezerwacja po stronie panelu pilota kończy się komunikatem potwierdzającym, natomiast endpointy backendowe odpowiedzialne za rezerwacje mają charakter przygotowanych zaślepek.

12 Role i uprawnienia użytkowników

W systemie przewidziano dwie główne role użytkowników: pilota oraz administratora. Rola użytkownika jest przechowywana w bazie danych w modelu `User`, a następnie przekazywana do frontendu po poprawnym logowaniu. Na jej podstawie aplikacja decyduje, do którego panelu przekierować użytkownika.

Pilot ma dostęp do panelu pilota, w którym może przeglądać dostępne statki powietrzne, sprawdzić ich status, cenę za minutę lotu oraz wybrać datę planowanego lotu. Administrator ma dostęp do panelu administracyjnego, w którym może przeglądać flotę oraz aktualizować stawki minutowe przypisane do modeli statków powietrznych.

Dostęp do widoków frontendu jest ograniczany za pomocą mechanizmu Navigation Guards w Vue Router. Użytkownik niezalogowany jest przekierowywany do strony logowania, a użytkownik bez roli administratora nie może wejść do panelu administracyjnego. Analogiczna kontrola dostępu występuje po stronie backendu: część endpointów wymaga poprawnego tokenu JWT, a endpoint aktualizacji cennika wymaga dodatkowo roli administratora.

Tabela 5: Macierz uprawnień użytkowników systemu E-Chronometraż

Funkcja systemu	Pilot	Administrator
Logowanie do systemu	Tak	Tak
Dostęp do panelu pilota	Tak	Tak
Podgląd listy statków powietrznych	Tak	Tak
Podgląd cennika	Tak	Tak
Wybór daty planowanego lotu	Tak	Tak
Edycja cennika	Nie	Tak
Dostęp do panelu administratora	Nie	Tak
Aktualizacja stawek minutowych	Nie	Tak

13 Opis API aplikacji

Backend aplikacji udostępnia API typu REST pod adresem bazowym `http://localhost:8080/api`. Komunikacja odbywa się przez HTTP, a dane są przesyłane w formacie JSON. Do ochrony

wybranych zasobów wykorzystywany jest token JWT przekazywany w nagłówku **Authorization** w formacie **Bearer <token>**.

Tabela 6: Najważniejsze endpointy API systemu

Metoda	Endpoint	Dostęp	Opis
POST	/api/auth/login	Publiczny	Logowanie użytkownika i wygenerowanie tokenu JWT.
GET	/api/airplanes	Pilot/Admin	Pobranie listy statków powietrznych wraz ze stawką minutową.
GET	/api/pricing	Pilot/Admin	Pobranie aktualnego cennika.
PUT	/api/pricing	Admin	Aktualizacja stawek minutowych.
GET	/api/reservations	Pilot/Admin	Przygotowany endpoint do pobierania rezerwacji.
POST	/api/reservations	Pilot/Admin	Przygotowany endpoint do tworzenia rezerwacji.

Endpoint `login` sprawdza dane logowania w bazie danych. Po poprawnym logowaniu backend zwraca dane użytkownika oraz token JWT ważny przez 2 godziny. Token jest następnie zapisywany po stronie frontendu w `localStorage` i używany przy kolejnych zapytaniach.

Endpoint `airplanes` pobiera z bazy listę statków powietrznych oraz cennik. Następnie backend łączy te dane w jedną odpowiedź, dzięki czemu frontend otrzymuje model, znak rejestracyjny, status oraz cenę za minutę lotu.

Endpoint `/api/pricing` w metodzie PUT umożliwia administratorowi aktualizację cennika. Obecna implementacja usuwa poprzednie rekordy cennika i zapisuje nowe wartości przesłane z panelu administracyjnego.

14 Przepływy działania systemu

14.1 Logowanie użytkownika

Użytkownik wprowadza login i hasło w formularzu logowania. Frontend wysyła zapytanie `POST /api/auth/login` do backendu. Backend sprawdza dane w bazie danych i w przypadku poprawnego logowania generuje token JWT. Frontend zapisuje token oraz dane użytkownika w `Pinia` i `localStorage`. Następnie użytkownik zostaje przekierowany do panelu zgodnego z jego rolą: administrator do panelu admina, a pilot do panelu pilota.

14.2 Przegląd statków powietrznych przez pilota

Po wejściu do panelu pilota aplikacja pobiera listę statków powietrznych z endpointu `GET /api/airplanes`. Zapytanie zawiera token JWT. Backend weryfikuje token, pobiera dane samolotów i cennika z bazy, a następnie zwraca połączoną listę. Frontend prezentuje użytkownikowi karty statków powietrznych zawierające model, znak rejestracyjny, status oraz cenę za minutę lotu.

14.3 Wybór terminu lotu

Pilot wybiera datę lotu z komponentu kalendarza. Jeżeli statek powietrzny ma status `available`, przycisk rezerwacji jest aktywny. Po kliknięciu przycisku aplikacja wyświetla komunikat potwierdzający wybór samolotu i daty. W obecnej wersji MVP operacja ta nie zapisuje jeszcze rezerwacji w bazie danych.

14.4 Aktualizacja cennika przez administratora

Administrator przechodzi do panelu administracyjnego. Widok pobiera listę statków powietrznych wraz z aktualnymi stawkami. Administrator może zmienić stawkę minutową w tabeli i zapisać zmiany. Frontend wysyła zapytanie `PUT /api/pricing` z tokenem JWT. Backend sprawdza poprawność tokenu oraz rolę administratora, a następnie aktualizuje dane cennika w bazie.

15 Walidacja danych i obsługa błędów

Aplikacja zawiera podstawową walidację po stronie frontendu oraz backendu. Formularz logowania wymaga podania loginu i hasła. W przypadku błędnych danych użytkownik otrzymuje komunikat o nieudanym logowaniu. Po stronie backendu błędny format danych wejściowych powoduje zwrócenie odpowiedzi `400 Bad Request`, a błędny login lub hasło powodują odpowiedź `401 Unauthorized`.

Dostęp do chronionych endpointów wymaga nagłówka `Authorization`. Brak tokenu, błędny format tokenu lub niepoprawny token skutkują odrzuceniem żądania. Dodatkowo endpoint aktualizacji cennika jest zabezpieczony rolą administratora. Jeżeli użytkownik nie ma odpowiednich uprawnień, backend zwraca odpowiedź `403 Forbidden`.

Po stronie interfejsu użytkownika przyciski rezerwacji są blokowane dla statków powietrznych, które nie mają statusu `available`. W panelu administratora pole stawki przyjmuje wartość liczbową i ma ustawioną minimalną wartość 0.

Tabela 7: Wybrane przypadki obsługi błędów

Sytuacja	Reakcja systemu
Brak loginu lub hasła	Formularz nie zostaje wysłany.
Błędne dane logowania	System wyświetla komunikat o błędzie logowania.
Brak tokenu JWT	Backend odrzuca dostęp do chronionych zasobów.
Brak roli administratora	Użytkownik jest przekierowywany lub backend zwraca błąd <code>403 Forbidden</code> .
Samolot w serwisie	Przycisk rezerwacji jest zablokowany.
Błąd aktualizacji cennika	System wyświetla komunikat o błędzie zapisu.

16 Testowanie systemu

W aktualnej wersji projektu nie przygotowano automatycznych testów jednostkowych ani integracyjnych. Weryfikacja działania systemu odbywa się manualnie poprzez urucho-

mienie aplikacji, wykonanie głównych scenariuszy użytkownika oraz sprawdzenie reakcji interfejsu i backendu.

Testy manualne obejmują logowanie użytkownika, przekierowanie do odpowiedniego panelu, ochronę tras, pobieranie listy statków powietrznych, wyświetlanie cennika, blokowanie niedostępnych statków powietrznych oraz aktualizację stawek przez administratora.

Tabela 8: Scenariusze testów manualnych

ID	Scenariusz testowy	Rezultat
T1	Logowanie jako <code>admin</code>	Przekierowanie do panelu administratora.
T2	Logowanie jako <code>pilot</code>	Przekierowanie do panelu pilota.
T3	Wejście niezalogowanego użytkownika na <code>/pilot</code>	Przekierowanie do logowania.
T4	Wejście pilota na <code>/admin</code>	Odmowa dostępu i przekierowanie.
T5	Pobranie listy statków powietrznych	Wyświetlenie kart samolotów.
T6	Próba rezerwacji samolotu w serwisie	Przycisk rezerwacji jest zablokowany.
T7	Zmiana stawki przez administratora	Cennik zostaje zaktualizowany.
T8	Żądanie API bez tokenu	Backend zwraca błąd autoryzacji.

17 Instrukcja uruchomienia projektu

Do uruchomienia projektu wymagane są: Go, Node.js z npm oraz PostgreSQL. Baza danych powinna mieć nazwę `aeroclub`. Tabele są tworzone automatycznie przez GORM przy pierwszym uruchomieniu backendu.

Pierwszym krokiem jest sklonowanie repozytorium:

```
git clone https://github.com/PWr-ISK-collaborative-project/Aeroklub_e-Chronometraz.git
cd Aeroklub_e-Chronometraz
```

Następnie należy przygotować bazę danych PostgreSQL o nazwie:

`aeroclub`

W katalogu `backend` należy utworzyć plik `.env` na podstawie `.env.example`:

```
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=twoj_password_od_postgres
DB_NAME=aeroclub
```

Następnie należy pobrać zależności backendu:

```
cd backend
go mod tidy
```

W osobnym kroku należy zainstalować zależności frontendu:

```
cd frontend  
npm install
```

Cały projekt można uruchomić z katalogu głównego poleceniem:

```
npm run dev
```

Polecenie to uruchamia równolegle backend oraz frontend. Backend działa pod adresem:

```
http://localhost:8080
```

Frontend działa pod adresem:

```
http://localhost:5173
```

Do uruchomienia wyłącznie backendu można użyć:

```
cd backend  
go run cmd/api/main.go
```

Przy pierwszym uruchomieniu backend wykonuje migracje bazy danych oraz dodaje dane testowe, jeżeli tabela użytkowników jest pusta.

18 Bezpieczeństwo systemu

System wykorzystuje tokeny JWT do autoryzacji użytkowników. Po poprawnym logowaniu backend generuje token zawierający identyfikator użytkownika, rolę oraz czas wygaśnięcia. Token jest ważny przez 2 godziny. Frontend przechowuje token w pamięci lokalnej i dołącza go do zapytań kierowanych do chronionych endpointów API.

Po stronie backendu za ochronę zasobów odpowiada middleware `RequireAuth`, który sprawdza obecność oraz poprawność tokenu JWT. Dodatkowo middleware `RequireAdmin` ogranicza dostęp do wybranych operacji wyłącznie dla użytkowników z rolą admin.

System ma skonfigurowaną politykę CORS, która pozwala na komunikację backendu z frontendem uruchomionym lokalnie na porcie 5173. Ogranicza to przypadkowe zapytania z innych źródeł w środowisku deweloperskim.

19 Decyzje projektowe i uzasadnienie technologii

Projekt został wykonany jako aplikacja webowa, ponieważ taki model pozwala użytkownikom korzystać z systemu bez instalowania dodatkowego oprogramowania. Jest to szczególnie korzystne dla aeroklubów, gdzie z aplikacji mogą korzystać różne osoby: piloci, administratorzy i osoby obsługujące flotę.

Frontend został przygotowany z użyciem Vue.js 3 oraz TypeScript. Vue umożliwia szybkie tworzenie reaktywnych interfejsów, a TypeScript ogranicza liczbę błędów wynikających z niejednoznacznych typów danych. Do zarządzania stanem użyto Pinia, ponieważ

pozwała wygodnie przechowywać dane użytkownika, token oraz dane pobierane z API. Vue Router odpowiada za routing oraz ochronę widoków.

Backend został napisany w Go z użyciem frameworka Gin. Go jest językiem dobrze dopasowanym do tworzenia prostych i wydajnych usług HTTP, a Gin upraszcza definiowanie tras, obsługę żądań oraz middleware. Do komunikacji z bazą danych zastosowano GORM, który ułatwia mapowanie modeli na tabele oraz automatyczne migracje.

Jako bazę danych wybrano PostgreSQL, ponieważ dane systemu mają naturalnie relacyjny charakter. Użytkownicy, statki powietrzne, cenniki oraz rezerwacje mogą być reprezentowane jako powiązane tabele. Komunikacja między frontendem i backendem została oparta o REST API, ponieważ jest to proste i czytelne rozwiązanie wystarczające dla zakresu MVP.

20 Podsumowanie realizacji wymagań

Aktualna wersja projektu realizuje podstawowe wymagania związane z logowaniem, podziałem ról, wyświetlaniem floty oraz zarządzaniem cennikiem. Część dotycząca rezerwacji została przygotowana od strony interfejsu użytkownika, jednak nie jest jeszcze w pełni zapisywana w bazie danych.

Tabela 9: Status realizacji wymagań funkcjonalnych

ID	Wymaganie	Status	Uwagi
F1	Logowanie i autoryzacja ról	Zrealizowane częściowo	JWT i role działają, hasła wymagają haszowania.
F2	Interaktywny kalendarz dostępności	Zrealizowane częściowo	Dostępny wybór daty, brak pełnej weryfikacji dostępności z bazy.
F3	Rezerwacja wolnego terminu	Zrealizowane częściowo	Frontend pokazuje potwierdzenie, backend ma zaślepki.
F4	Zarządzanie flotą i cennikiem	Zrealizowane częściowo	Dostępna edycja cennika; zarządzanie flotą ograniczone do podglądu.
F5	Automatyczne wyliczanie kosztu lotu	Niezrealizowane w pełni	System prezentuje stawki minutowe, ale nie zapisuje rozliczonego lotu.

Wymagania нефункционалне zostały zrealizowane częściowo. System posiada podstawową autoryzację JWT, podział ról i konfigurację CORS. Wydajność aplikacji w środowisku lokalnym jest wystarczająca dla zakresu MVP. Dostępność 24/7 nie została zaimplementowana, ponieważ projekt nie został wdrożony na środowisko produkcyjne.

Tabela 10: Status realizacji wymagań niefunkcjonalnych

ID	Wymaganie	Status	Uwagi
NF1	Bezpieczeństwo: hasła, JWT i role użytkowników	Zrealizowane częściowo	JWT i role działają; haszowanie haseł wymaga dopracowania.
NF2	Wydajność: szybkie ładowanie interfejsu	Zrealizowane częściowo	W środowisku lokalnym aplikacja działa płynnie.
NF3	Dostępność: system dostępny 24/7	Niezrealizowane	Brak wdrożenia produkcyjnego.

21 Możliwości dalszego rozwoju

Największy potencjał dalszego rozwoju systemu E-Chronometraż wiąże się z rozszerzeniem modułu rozliczeń. System może zostać wzbogacony o możliwość wpisywania rzeczywistego czasu lotu, automatyczne obliczanie kosztu na podstawie stawki minutowej oraz zapisywanie historii rozliczeń. Dzięki temu aplikacja mogłaby stać się nie tylko narzędziem do planowania, ale również centralnym miejscem obsługi finansowej lotów wykonywanych w aeroklubie.

Dużą wartość dla użytkowników końcowych mogłoby wnieść dodanie historii lotów pilota. Pilot po zalogowaniu mógłby przeglądać swoje wcześniejsze loty, koszty, wykorzystane statki powietrzne oraz zaplanowane rezerwacje. Z perspektywy administratora przydatnym rozszerzeniem byłby panel raportów, umożliwiający analizę wykorzystania floty, miesięcznych rozliczeń oraz aktywności pilotów.

System można również rozwijać w kierunku lepszej obsługi operacyjnej aeroklubu. Przykładowymi funkcjami są: status techniczny statków powietrznych, automatyczne blokowanie rezerwacji dla maszyn czasowo niedostępnych, moduł instruktorów, powiadomienia e-mail o rezerwacjach oraz przypomnienia o zaplanowanych lotach. Takie funkcje mogłyby znacząco zwiększyć praktyczną użyteczność aplikacji w codziennej pracy aeroklubu.

W dalszej perspektywie E-Chronometraż może zostać zintegrowany z dodatkowymi usługami, takimi jak eksport raportów do plików PDF lub CSV, integracja z systemami księgowymi, generowanie zestawień miesięcznych oraz rozbudowany panel administracyjny. Dzięki modułowej architekturze aplikacja może być stopniowo rozwijana bez konieczności przebudowy całego systemu.

22 Deklaracja wykorzystania narzędzi AI

Narzędzie	Zastosowanie	Zakres	Uwagi
Google Gemini	Wsparcie PM i techniczne	Dokumentacja, struktura projektu	Pomoc w formacie LaTeX i planowaniu prac.

Tabela 11: Deklaracja użycia narzędzi AI.